

Relazione – Laboratori di Data Science (Discesa del gradiente e Regressione Polinomiale)

Diego Quarantani

Aprile 2026

Questa relazione raccoglie lo svolgimento degli esercizi del **Foglio 2 – Ottimizzazione** del corso di Data Science (Laurea in Fisica, Alma Mater Studiorum – Università di Bologna, a.a. 2025/2026). Per tutto il codice, si rimanda alla repository GitHub: **Laboratori-Data-Science**

1. Design del codice

1.1. Generazione dei dati

```
noise_amp = 0.2
n_samples = 200
eta = 0.5

x = np.random.uniform(0, 1, n_samples)
noise = np.random.uniform(-noise_amp, noise_amp, n_samples)
y = np.sin(2 * np.pi * x) + noise

x_true = np.linspace(0, 1, 500)
y_true = np.sin(2 * np.pi * x_true)

degree = 9
degree += 1
coeffs = np.random.uniform(-0.5, 0.5, degree)
```

I dati sono stati generati utilizzando le funzionalità offerte dalla libreria **numpy**, facendo uso esclusivo di array. Tale scelta permette di eseguire le operazioni in forma vettorializzata, con un conseguente accesso alla *parallelizzazione* del carico computazionale rispetto a un'implementazione basata su iterazioni esplicite. Questa modalità di gestione dei dati garantisce una buona scalabilità del codice.

In tutte le esecuzioni presentate in questa relazione sono stati utilizzati 200 punti generati secondo la procedura illustrata sopra: i valori di x sono estratti casualmente da una distribuzione uniforme nell'intervallo $[0, 1]$, mentre i corrispondenti valori di y sono ottenuti a partire dalla funzione seno, con l'aggiunta di un termine di rumore. Quando non specificato, il valore del parametro **eta** (ossia il *learning rate*) è stato fissato a 0.5.

1.2. Modello di regressione polinomiale

```
def polynomial_model(coeffs: np.ndarray, x: np.ndarray):
    powers = x[:, None] ** np.arange(len(coeffs))
    return powers @ coeffs
```

La funzione riportata sopra implementa il modello di regressione polinomiale. Essa riceve in ingresso un array di coefficienti e un array di valori x , e restituisce l'array dei corrispondenti valori predetti y , associati uno a uno agli elementi di input.

L'istruzione `x[:, None]` trasforma il vettore x in un array colonna, mentre `np.arange(len(coeffs))` genera il vettore degli esponenti del polinomio, cioè `[0, 1, ..., g]`, dove g rappresenta il grado del polinomio. L'operatore `**` viene quindi utilizzato per elevare ciascun valore di x a tutte le potenze richieste, costruendo così la matrice delle potenze:

$$\text{powers} = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^g \\ 1 & x_2 & x_2^2 & \dots & x_2^g \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^g \end{pmatrix}$$

A questo punto, il prodotto riga-colonna tra `powers` e `coeffs` consente di calcolare simultaneamente il valore del polinomio in tutti i punti considerati.

Questo approccio richiede un maggiore utilizzo di memoria rispetto a una versione iterativa, poiché costruisce esplicitamente la matrice delle potenze, ma risulta generalmente più efficiente in termini di tempo di esecuzione grazie all'ottimizzazione delle operazioni vettoriali.

1.3. Discesa del gradiente

La formula analitica per il calcolo del gradiente, nel caso di funzione costo MSE e modello di regressione polinomiale, è la seguente:

$$\nabla_{\mathbf{W}} J(\mathbf{W}) = -\frac{1}{N} \begin{pmatrix} \sum_{i=1}^N (y_i - \hat{y}_i) x_i^0 \\ \sum_{i=1}^N (y_i - \hat{y}_i) x_i^1 \\ \sum_{i=1}^N (y_i - \hat{y}_i) x_i^2 \\ \vdots \\ \sum_{i=1}^N (y_i - \hat{y}_i) x_i^d \end{pmatrix}$$

L'implementazione in Python è la seguente:

```
def gradient_cost_function(
    coeffs: np.ndarray, x: np.ndarray, y: np.ndarray
) → np.ndarray:
    predictions = polynomial_model(coeffs, x)
    errors = predictions - y

    X_poly = x[:, None] ** np.arange(len(coeffs))
    gradient = errors @ X_poly / len(x)
    return gradient
```

Nella prima versione del codice il gradiente veniva calcolato mediante un ciclo for sui vari esponenti del polinomio; la successiva riformulazione matriciale ha permesso di ottenere un miglioramento prestazionale di circa un fattore 2, raddoppiando il numero di iterazioni eseguibili al secondo.

L'aggiornamento iterativo dei coefficienti nel main del programma tramite discesa del gradiente è poi implementato nel seguente ciclo:

```

with tqdm(range(40000), desc="Ottimizzazione", unit="iter") as t:
    for i in t:
        grad = gradient_cost_function(coeffs, x, y)
        cost = cost_function(coeffs, x, y)
        t.set_postfix({"Costo": f"{cost:.6e}"})
        coeffs -= eta * grad

```

La prima riga di questo frammento di codice consente di visualizzare una barra di avanzamento durante il processo di ottimizzazione, rendendo più agevole il monitoraggio dell'esecuzione. Inoltre, permette di osservare in tempo reale l'andamento del valore della funzione costo (Sezione 1.4), verificando che esso diminuisca effettivamente nel corso del training.

1.4. Funzione costo

La funzione costo è implementata come segue:

```

def cost_function(coeffs: np.ndarray, x: np.ndarray, y: np.ndarray) → float:
    predictions = polynomial_model(coeffs, x)
    errors = predictions - y
    return np.mean(errors**2) / 2

```

Si calcola il vettore `predictions` applicando il modello polinomiale ai dati di input `x`. Successivamente, il vettore `errors` contiene le differenze tra le predizioni e i valori osservati `y`. Infine, la funzione restituisce la media dei quadrati degli errori divisa per 2, in accordo con la definizione della funzione di costo richiesta.

1.5. Mini batch code

```

def get_batches(x, y, batch_size):
    indices = np.arange(len(x))
    np.random.shuffle(indices)

    for i in range(0, len(x), batch_size):
        batch_idx = indices[i : i + batch_size]
        yield x[batch_idx], y[batch_idx]

epochs = []
times = []
conv_cost = 6.7e-3
coeffs_init = np.random.uniform(-0.5, 0.5, degree)

for batch_size in range(1, 101):
    start_time = time.perf_counter()

    epoch = 0
    coeffs = coeffs_init.copy()
    cost = cost_function(coeffs, x, y)

    while cost > conv_cost:
        epoch += 1

```

```

    for x_batch, y_batch in get_batches(x, y, batch_size):
        grad = gradient_cost_function(coeffs, x_batch, y_batch)
        coeffs -= eta * grad

    cost = cost_function(coeffs, x, y)

epochs.append(epoch)
end_time = time.perf_counter()
times.append(end_time - start_time)

```

L'implementazione della variante Mini-batch Stochastic Gradient Descent è riportata nel codice precedente. L'elemento centrale è la funzione `get_batches(x, y, batch_size)`, che ha il compito di suddividere il dataset in sottoinsiemi casuali di dimensione controllata.

Il funzionamento può essere descritto nei seguenti passaggi:

- si considera un array di indici del dataset che viene rimescolato;
- l'istruzione `range(0, len(x), batch_size)` suddivide implicitamente gli indici in blocchi consecutivi della dimensione desiderata;
- tramite selezione indicizzata vengono estratti i corrispondenti sottoinsiemi di x e y ;
- infine, la keyword `yield` consente di restituire un batch alla volta senza terminare l'esecuzione della funzione, preservandone lo stato interno tra una chiamata e la successiva.

Quest'ultimo aspetto è particolarmente utile in questo contesto, poiché permette di generare i mini-batch in modo progressivo durante l'iterazione, senza doverli memorizzare tutti simultaneamente in una struttura dati separata.

All'interno del ciclo principale, l'algoritmo segue una logica del tutto analoga a quella della discesa del gradiente standard. Sono inoltre presenti alcune istruzioni aggiuntive per misurare il tempo necessario al raggiungimento della convergenza.

2. Risultati

2.1. Monitoraggio della convergenza

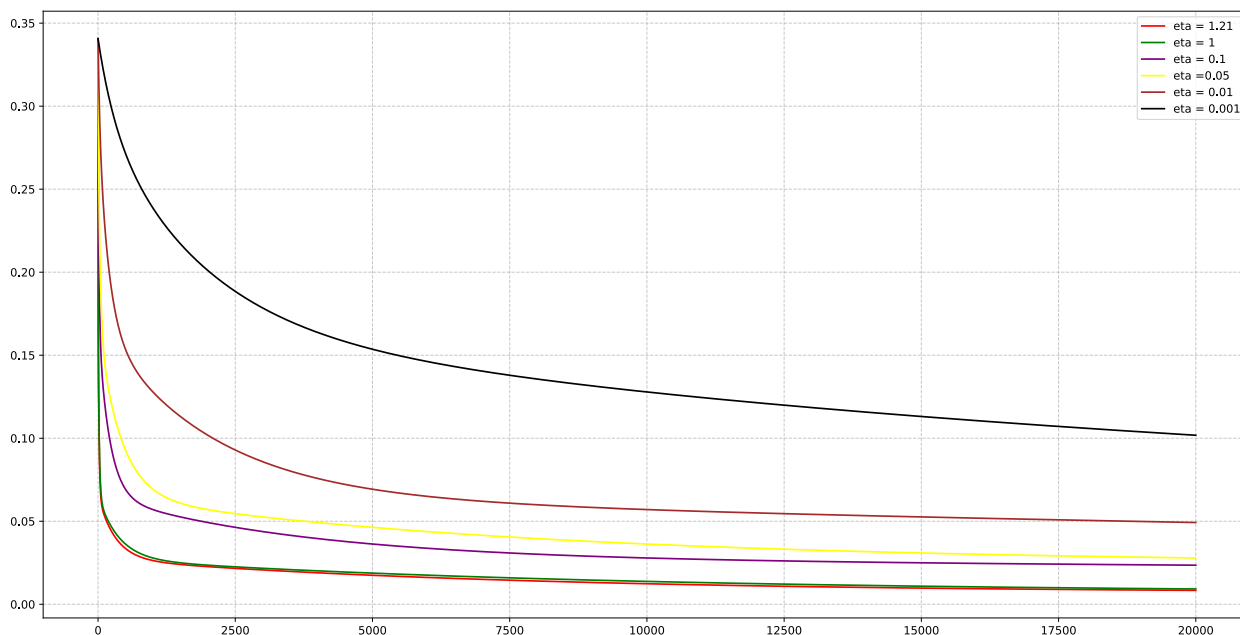


Figura 1: Andamento della funzione costo in funzione del numero di iterazioni, fino a un massimo di 20 000. Le curve, rappresentate con colori diversi, corrispondono a differenti valori del *learning rate*. Per rendere il confronto significativo, i coefficienti del modello sono stati inizializzati agli stessi valori casuali per ogni valore di *eta*, così da far partire l'ottimizzazione dalla medesima condizione iniziale.

Come si può osservare in Figura 1, valori più elevati del *learning rate* portano generalmente a una diminuzione più rapida della funzione costo, consentendo di raggiungere valori più bassi in un numero inferiore di iterazioni. Questo comportamento è coerente con il fatto che, aumentando la dimensione del passo di aggiornamento, l'algoritmo percorre più velocemente la direzione di discesa.

Tuttavia, tale vantaggio è accompagnato da una minore stabilità numerica. Se il *learning rate* assume valori eccessivamente elevati, gli aggiornamenti dei coefficienti possono diventare troppo bruschi, compromettendo la convergenza dell'algoritmo.

In una seconda *run* del programma, riportata in Figura 2, è stato possibile evidenziare questo aspetto. In questo caso si osserva che alcuni valori di *eta*, pur risultando efficaci in altre condizioni iniziali, non garantiscono più la convergenza del metodo. In particolare, per $\eta = 1.21$ il processo di ottimizzazione risulta instabile e non converge correttamente.

Questo risultato mostra come la scelta del *learning rate* non dipenda esclusivamente dalla forma della funzione costo, ma anche dalla posizione iniziale nello spazio dei parametri. In generale, valori troppo elevati di *eta* possono compromettere la robustezza del processo di ottimizzazione, rendendo il comportamento dell'algoritmo fortemente sensibile alle condizioni iniziali.

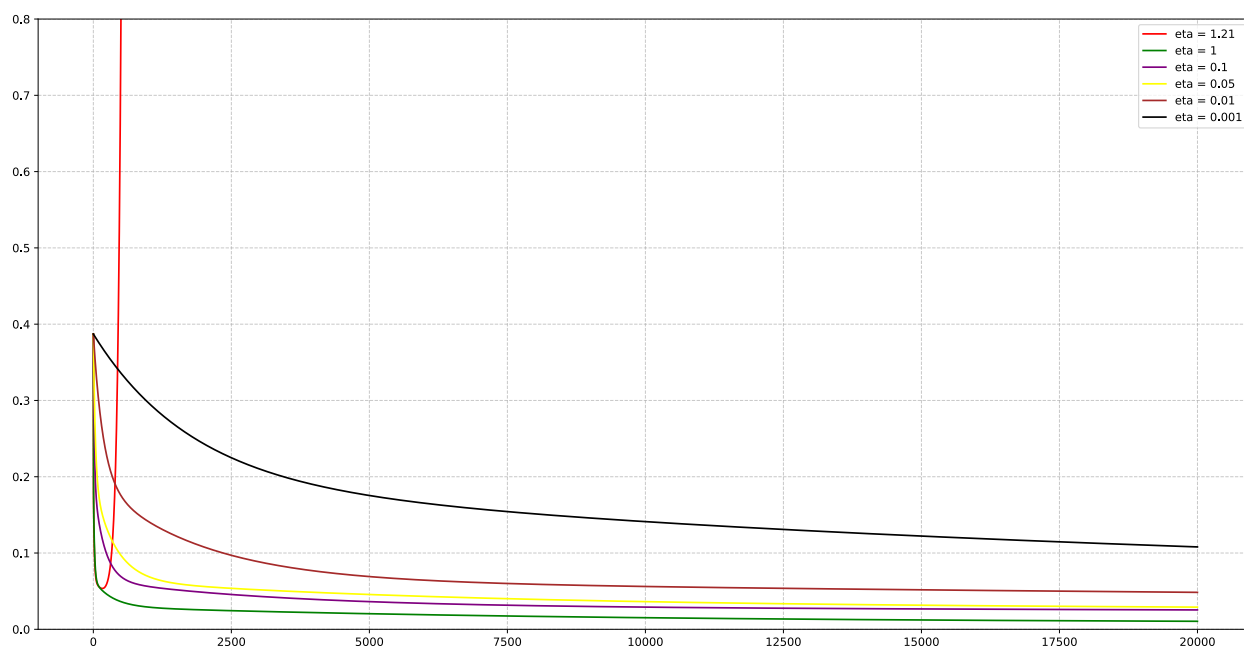


Figura 2: Secondo confronto dell'andamento della funzione costo al variare del learning rate, si evidenzia come **eta** alti siano più instabili e possano non convergere al minimo della funzione costo.

2.1.1. Fit della curva

In tutti i fit riportati in questa sezione è stato utilizzato un valore di **eta** = 0.5. Dalle prove preliminari è emerso che questo valore rappresenta un buon compromesso tra rapidità di convergenza e stabilità numerica, consentendo all'algoritmo di raggiungere risultati soddisfacenti in tempi contenuti senza introdurre, nella maggior parte dei casi, comportamenti instabili.

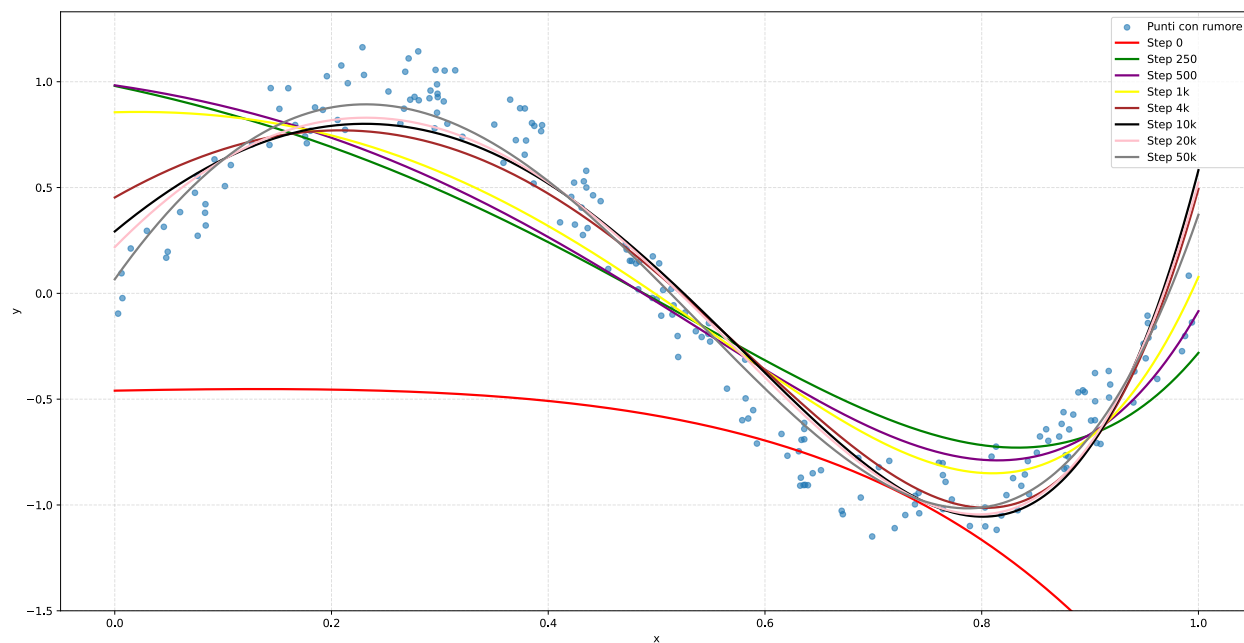


Figura 3: Risultato del fit polinomiale utilizzando un polinomio di grado 7.

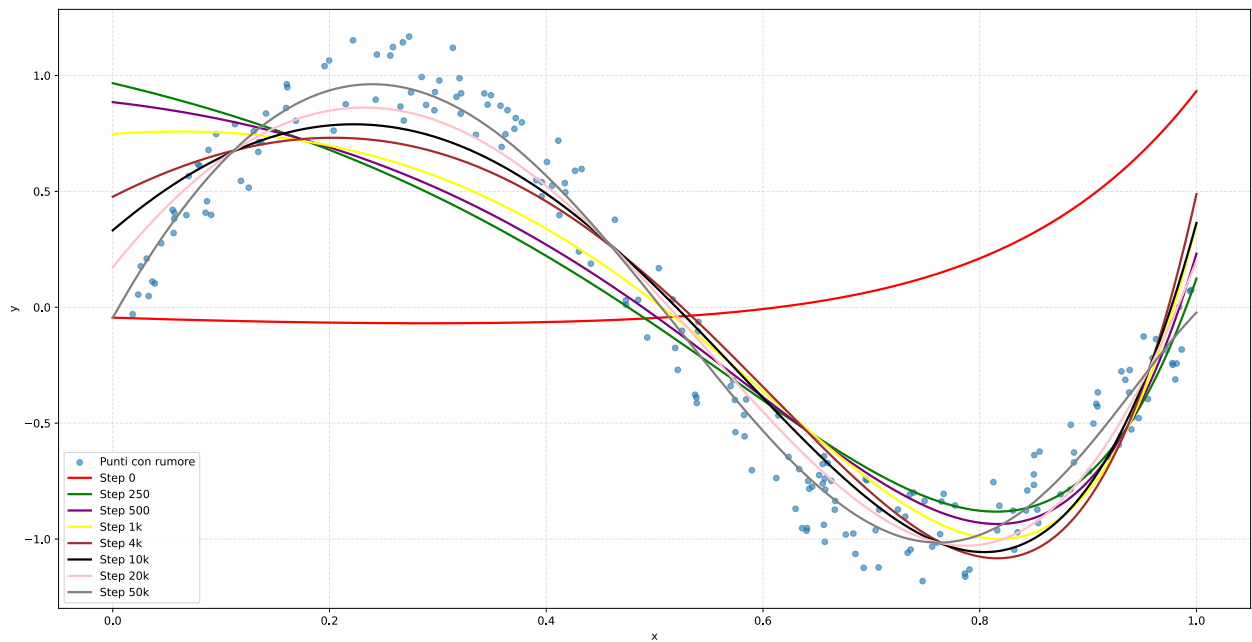


Figura 4: Risultato del fit polinomiale utilizzando un polinomio di grado 9.

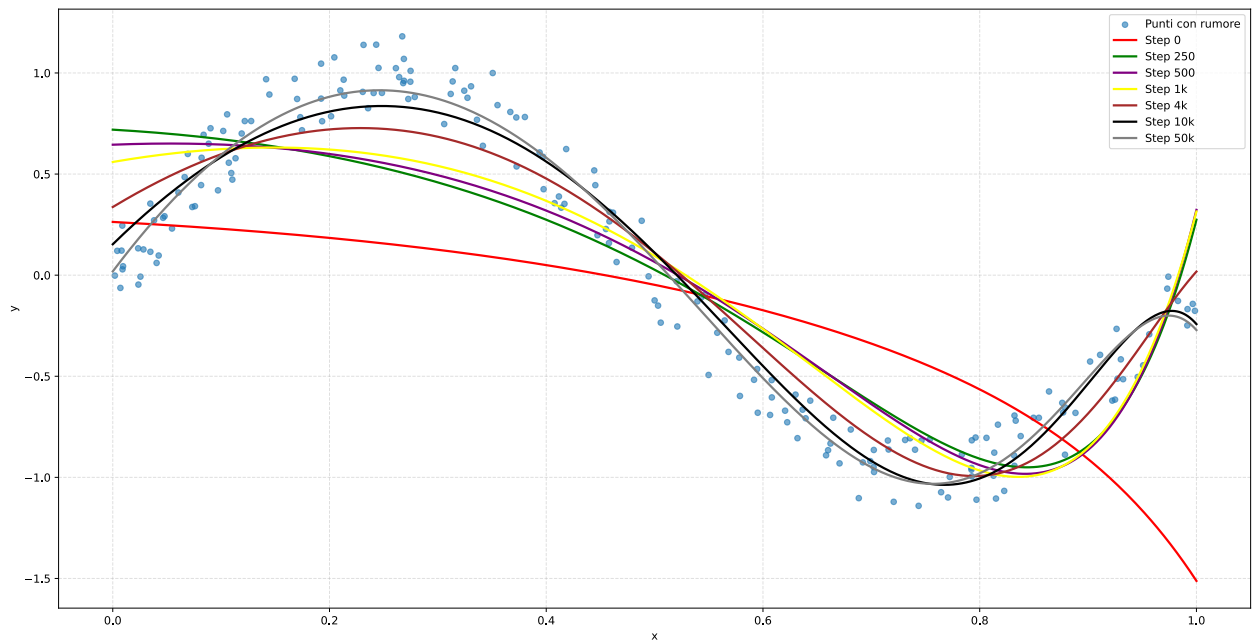


Figura 5: Risultato del fit polinomiale utilizzando un polinomio di grado 14.

Confrontando i risultati ottenuti per diversi gradi del polinomio, è possibile osservare come la complessità del modello influenzi la qualità dell'approssimazione.

Nel caso mostrato in Figura 3 (grado 7), il fit risulta già qualitativamente buono: la curva approssima in maniera ragionevole l'andamento generale dei dati.

In Figura 4 si osserva un lieve miglioramento della qualità del fit. La curva grigia segue in modo più fedele l'andamento dei punti sperimentali e della funzione sottostante. Tra i casi analizzati, questo è risultato essere uno dei più efficaci, riuscendo a migliorare l'adattamento senza introdurre fenomeni di *over-fitting*.

Aumentando la complessità del modello, quindi il grado del polinomio, come mostrato in Figura 5, si acquisisce una maggiore capacità di adattarsi ai dati. Tuttavia, iniziano a comparire i primi segnali di un comportamento eccessivamente aderente ai punti osservati. In particolare, nella parte finale della curva essa piega verso il basso avvicinandosi ad una zona lievemente più densa di punti, discostandosi

dal valore del seno originale, si tratta di un primo accenno di *over-fitting*. Per una discussione più approfondita di questo fenomeno si rimanda alla Sezione 2.2.

2.2. Over-fitting

Per mostrare in modo più evidente il fenomeno dell'*over-fitting*, è stato eseguito un ulteriore esperimento impostando il grado del polinomio a 300. Si è eseguita una *run* con circa 5 milioni di iterazioni, il risultato ottenuto è riportato in Figura 6.

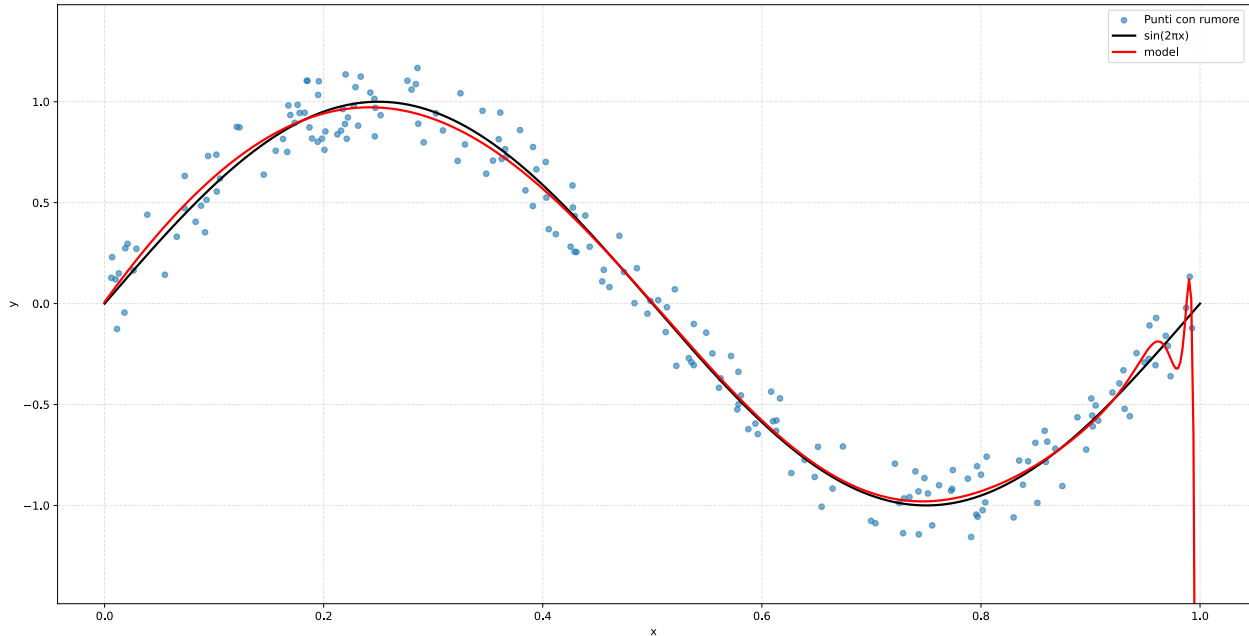


Figura 6: Esempio di *over-fitting* ottenuto utilizzando un polinomio di grado 300. Il valore finale della funzione costo è pari a 6.57×10^{-3} .

In questo caso il comportamento del modello evidenzia chiaramente i limiti di un'eccessiva complessità. Si osserva come nella regione finale il polinomio tenda a deformarsi sensibilmente pur di passare molto vicino agli ultimi punti campionati.

Questo comportamento permette effettivamente di ridurre ulteriormente il valore della funzione costo, ma a scapito della capacità del modello di rappresentare correttamente l'andamento generale della funzione generatrice.

2.3. Variante Mini-batch Stochastic Gradient Descent

Per confrontare l'effetto della dimensione del batch sul comportamento dell'ottimizzazione, è stata implementata anche la variante Mini-batch Stochastic Gradient Descent. In questa configurazione, il gradiente non viene calcolato sull'intero dataset a ogni aggiornamento, ma soltanto su sottoinsiemi casuali di dati di cardinalità `batch_size`.

Questo approccio modifica la dinamica della convergenza: batch più piccoli introducono una maggiore componente stocastica negli aggiornamenti, mentre batch più grandi rendono il processo più regolare.

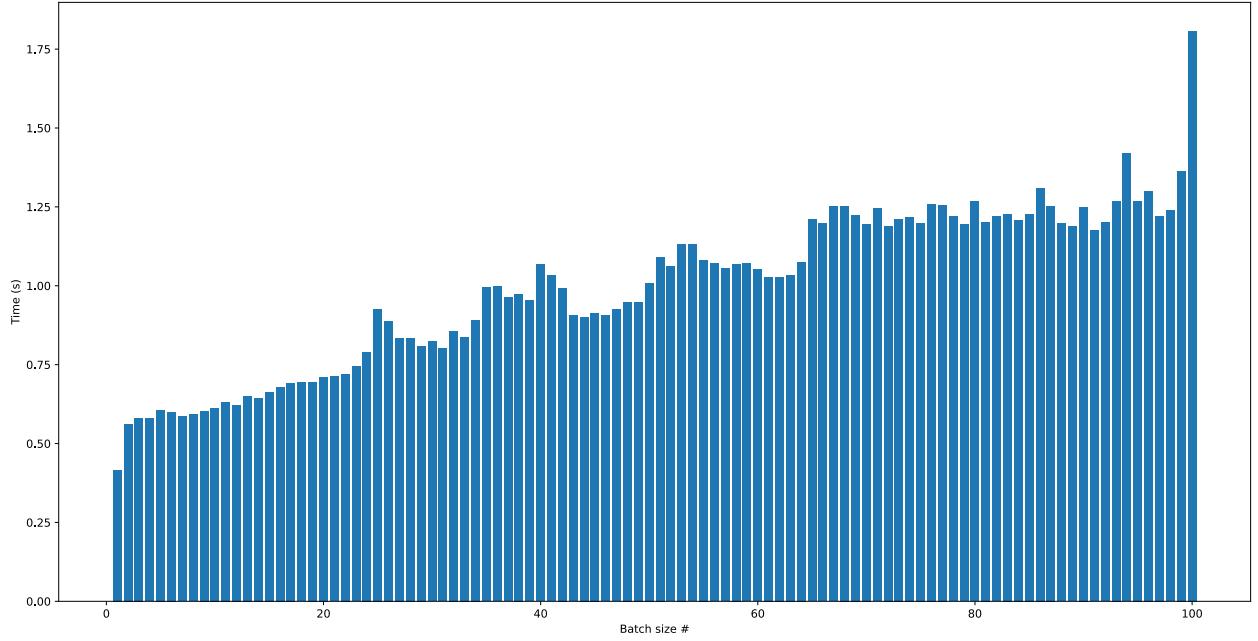


Figura 7: Tempo necessario per raggiungere la convergenza in funzione della dimensione del batch, utilizzando un polinomio di grado 9. Sull'asse delle x è riportato il valore di `batch_size`, mentre sull'asse delle y è riportato il tempo di esecuzione in secondi. La convergenza è stata definita come il raggiungimento di un valore della funzione costo inferiore a 6.7×10^{-3} . Il tempo minimo osservato è pari a 0.41 s, corrispondente a `batch_size` = 1, mentre il tempo massimo è pari a 1.83 s, corrispondente a `batch_size` = 100. I tempi riportati devono essere interpretati in modo qualitativo, poiché sono stati ottenuti tramite una semplice misurazione interna in Python e non mediante una procedura di benchmarking rigorosa.

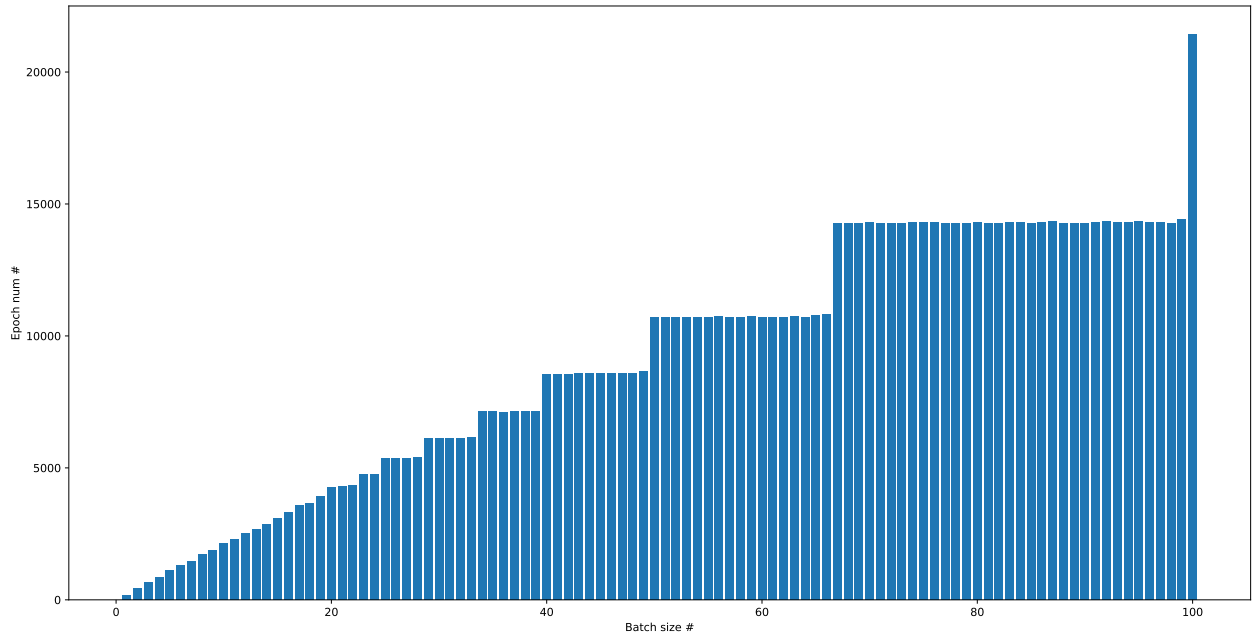


Figura 8: Numero di epoche necessarie per raggiungere la convergenza in funzione della dimensione del batch, utilizzando un polinomio di grado 9. Sull'asse delle x è riportato il valore di `batch_size`, mentre sull'asse delle y è riportato il numero di epoche richieste per ottenere un valore della funzione costo inferiore a 6.7×10^{-3} . Il numero minimo di epoche osservato è pari a 171, corrispondente a `batch_size` = 1, mentre il massimo è pari a 21431, corrispondente a `batch_size` = 100.

L'analisi dei tempi riportati in Figura 7 mostra un comportamento interessante al variare della dimensione del batch. Al netto delle piccole fluttuazioni osservabili tra barre vicine — attribuibili sia all'imprecisione intrinseca della misurazione dei tempi sia alla componente casuale introdotta dalla generazione dei batch — si osserva un andamento complessivamente crescente.

Il metodo che converge più rapidamente risulta essere quello con `batch_size = 1`, cioè il caso degenerare in cui ogni aggiornamento viene effettuato utilizzando un singolo punto del dataset. Questo risultato appare sospetto, poiché ci si aspetterebbe che un gradiente così rumoroso comprometta la stabilità e, più in generale, la convergenza del processo di ottimizzazione.

Le prestazioni peggiori si osservano invece per `batch_size = 100`, che corrisponde di fatto a una suddivisione del dataset in soli due batch.

In Figura 8 si osserva un incremento del numero di epoche all'aumentare di `batch_size`, con un andamento a gradoni. Tale comportamento è dovuto al fatto che batch più grandi riducono il numero di aggiornamenti effettuati per epoca; di conseguenza, per raggiungere la convergenza sono necessarie più epoche. La struttura a gradoni è anch'essa attesa, poiché il numero di batch per epoca varia in modo discreto: ogni volta che l'aumento di `batch_size` comporta una riduzione di un'unità nel numero totale di batch necessari a coprire i 200 punti del dataset, si osserva un salto nel numero di epoche richieste.

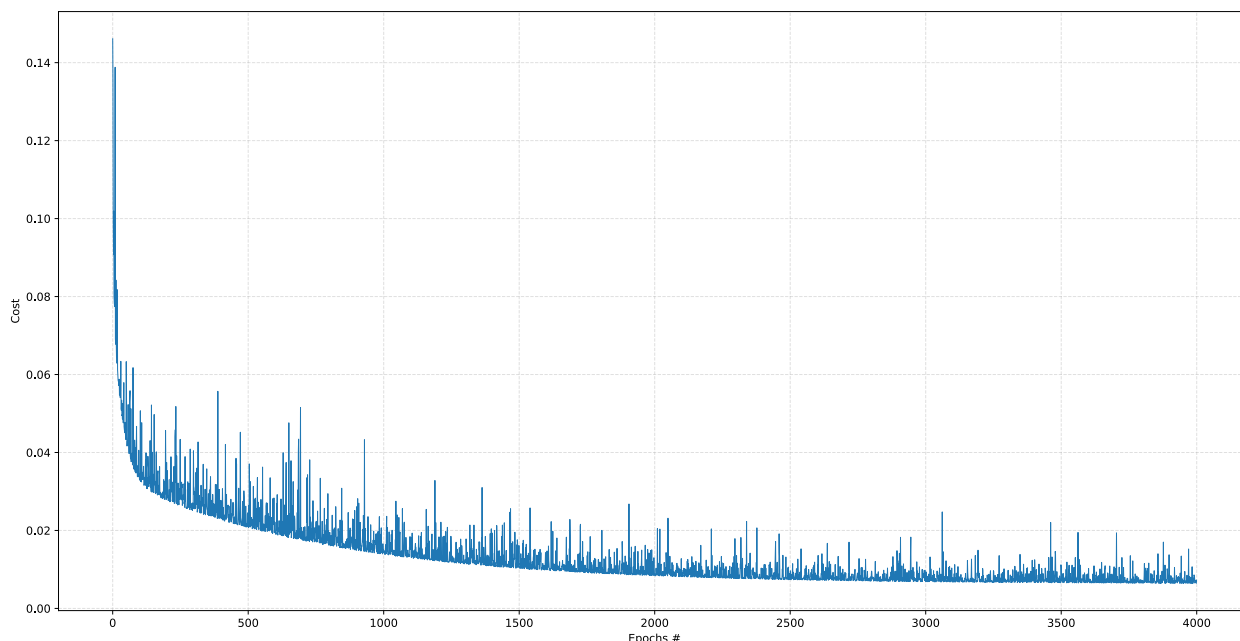


Figura 9: Andamento della funzione costo in funzione del numero di epoche nel caso `batch_size = 15`, utilizzando un polinomio di grado 9. Il grafico evidenzia il carattere rumoroso della convergenza tipico della discesa del gradiente stocastica su mini-batch.

Dal punto di vista qualitativo, i batch più piccoli tendono a rendere il processo di ottimizzazione più rumoroso, poiché ogni aggiornamento viene calcolato su un sottoinsieme ridotto di dati. Questo comportamento è ben visibile in Figura 9, dove, pur osservandosi una tendenza complessiva alla diminuzione della funzione costo, l'andamento risulta irregolare e soggetto a oscillazioni anche marcate.

Questo grafico permette anche di reinterpretare i risultati precedenti. Per come è stato implementato il criterio di arresto utilizzato per generare la Figura 7 — cioè interrompendo l'ottimizzazione non appena la funzione costo scendeva al di sotto di un valore target prefissato — il caso `batch_size = 1` non corrisponde a una convergenza stabile. Più precisamente, il processo può raggiungere in modo casuale un valore sufficientemente basso della funzione costo, senza che ciò implichi un effettivo assestamento della dinamica di ottimizzazione. In queste condizioni, l'algoritmo può infatti arrestarsi

in corrispondenza di minimi locali oppure su valori della funzione costo semplicemente bassi (che però potrebbero scendere ulteriormente). In un problema di fitting questo aspetto può talvolta essere riconosciuto visivamente osservando la curva ottenuta, mentre in contesti di machine learning più generali tale valutazione risulta meno immediata.